

数据通信与网络课程项目

从物理层到应用层的完整实现

团队成员:

- 欧阳倬历 (12310703)
- 徐浚丞 (12312423)

日期: 2025年12月

目录

- 项目概述 - 技术架构与目标
- Level 1: 点对点通信 - 物理层到数据链路层
- Level 2: 多主机通信 - 网络层与交换机
- Level 3: 扩展功能 - 传输层到应用层
- 核心代码讲解 - 关键模块实现
- 演示结果 - 实验验证与性能分析
- 总结与展望 - 技术亮点与改进方向

项目概述 - 技术架构

OSI 五层模型实现

应用层 (HTTP)	← Level 3: 应用协议
传输层 (ARQ)	← Level 3: 可靠传输
网络层 (MAC)	← Level 2: 寻址/转发
数据链路层 (Frame)	← Level 1: 分片/校验
物理层 (OOK)	← Level 1: 调制/解调
Cable (信道)	← 课程提供的物理介质

项目概述 - 三个Level

Level 1: 点对点通信 ✓

- **目标:** 两台主机通过 Cable 成功传输字符串
- **核心:** OOK调制、分片机制、校验和
- **挑战:** 噪声环境下的可靠传输

Level 2: 多主机通信 ✓

- **目标:** 星型拓扑下多主机通信
- **核心:** MAC地址、交换机学习算法
- **挑战:** 广播风暴优化

Level 3: 扩展功能 ✓

- **实现:** 传输层、信道编码、应用层、多调制、并发
- **亮点:** Stop-and-Wait ARQ、Hamming码、HTTP协议

Level 1: 点对点通信

物理层 → 数据链路层

任务 1.1: 比特流传输

核心技术: OOK 调制

原理:

- 比特 1 → 高电平 (振幅 = 1.0)
- 比特 0 → 零电平 (振幅 = 0.0)
- 每比特使用 **100 个采样点**

自适应阈值解调:

```
threshold = (max(signal) + min(signal)) / 2
```

关键优势:

- 实现简单，硬件成本低
- 在低噪声环境下性能优异

任务 1.1: 实验结果

原始消息: "Hello"

比特流: [0,1,0,0,1,0,0,0, 0,1,1,0,0,1,0,1, ...]

比特数: 40 bits (每字符8位)

OOK调制后: 4000 个采样点 (每bit 100个采样)

信号范围: [0.0, 1.0]

通过Cable传输 (噪声=0.01)

接收信号范围: [-0.03, 0.94]

恢复消息: "Hello"

传输成功: True

关键代码讲解:

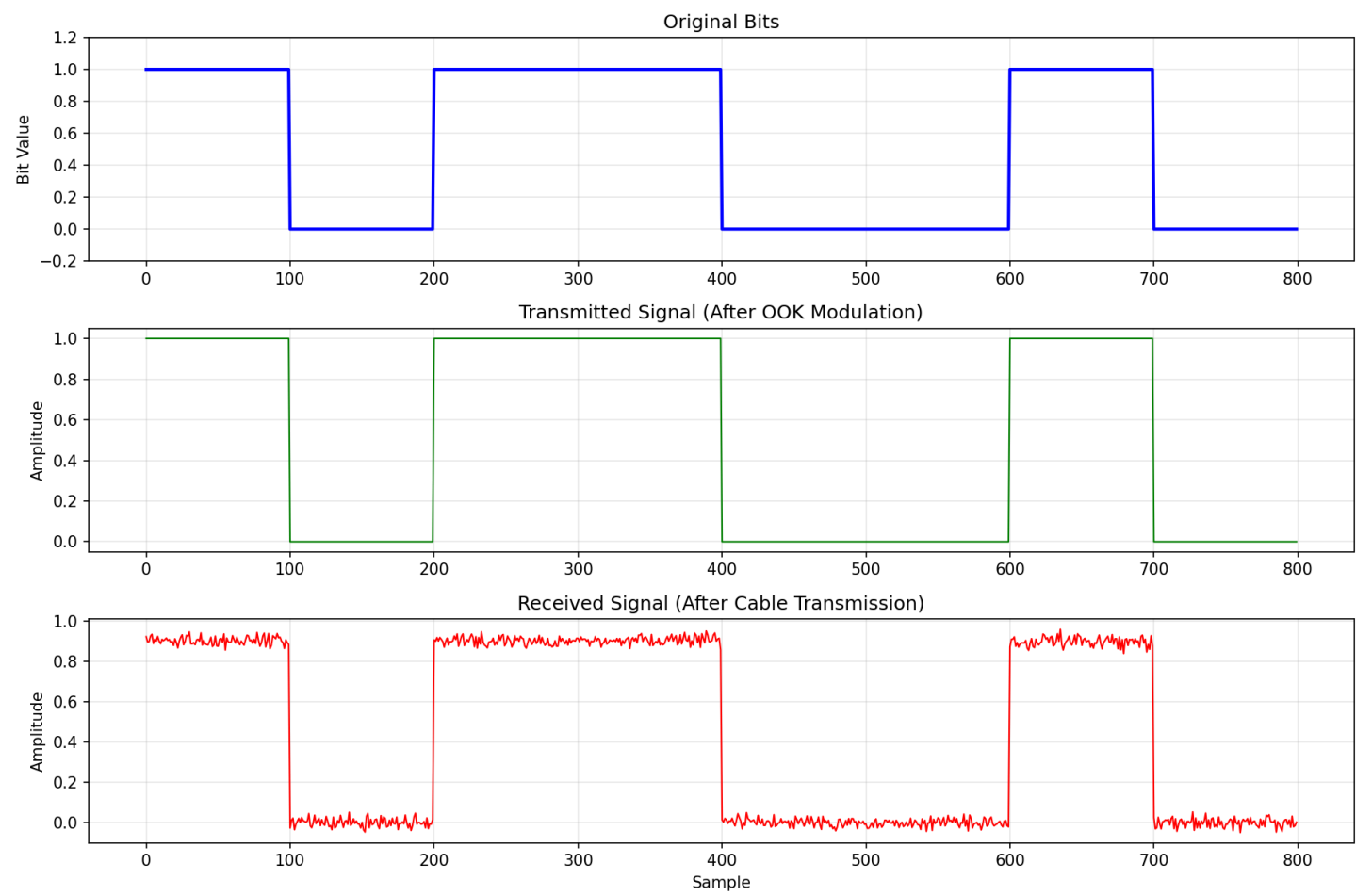
OOK 调制函数 - 将数字比特转换为模拟信号

```
def modulate_ook(bits: List[int], samples_per_bit=100) -> np.ndarray:
    signal = np.zeros(len(bits) * samples_per_bit)
    for i, bit in enumerate(bits):
        if bit == 1:
            signal[i*samples_per_bit:(i+1)*samples_per_bit] = 1.0
    return signal
```

工作原理:

- 创建空信号数组, 长度 = 比特数 × 100采样点
- 遍历每个比特: 比特1 → 填充100个高电平(1.0)
- 比特0 → 保持零电平(0.0)
- 输出连续的模拟信号波形

任务 1.1: 信号波形可视化



图片说明 - 比特传输全过程:

- **上图:** 原始比特序列（数字信号）
- **中图:** OOK调制后的发送信号（模拟信号，矩形波）
- **下图:** 通过Cable接收的信号（含噪声和衰减）

关键观察: 噪声使接收信号产生抖动，但100采样点平均可有效恢复比特

任务 1.2: 消息分片传输

分片机制 (MTU = 64 字节)

问题: 长消息超过物理层处理能力

解决方案: PacketSlicer 类

```
class PacketSlicer:
    def slice(self, data: bytes, max_size: int = 64) -> List[bytes]:
        """将消息切分为固定大小的分片"""
        return [data[i:i+max_size]
                for i in range(0, len(data), max_size)]

    def reassemble(self, slices: List[bytes]) -> bytes:
        """重组分片为完整消息"""
        return b''.join(slices)
```

代码讲解:

- slice(): 使用切片操作 data[i:i+max_size] 将字节流分割
- 列表推导式遍历, 步长为 max_size, 生成分片列表
- reassemble(): 使用 b''.join() 拼接字节分片
- **设计思想:** 类似TCP的MSS (最大段大小) 机制

任务 1.2: 帧格式设计

Preamble	Length	Payload	Checksum	Postamble
8 bits	16 bits	variable	8 bits	8 bits

实验结果:

原始消息长度：300 字节

分片数量：5

- 分片 0: 64 字节
- 分片 1: 64 字节
- 分片 2: 64 字节
- 分片 3: 64 字节
- 分片 4: 44 字节

传输成功：True

数据匹配：True

任务 1.3: 噪声环境测试

100 采样点平均效应

有效噪声 = 实际噪声 / $\sqrt{100}$ = 实际噪声 / 10

测试结果:

测试消息: "Hello World"

噪声σ	有效σ	接收结果	误码率

0.5	0.05	Hello World	0.00% [OK]
1.0	0.10	Hello World	0.00% [OK]
1.2	0.12	Hello World	0.00% [OK]
1.5	0.15	Hello World	0.11% [OK]
1.8	0.18	Hello World	0.45% [OK]
2.0	0.20	Hello World	1.36% [OK]

结论: 采样点平均显著降低噪声影响

任务 1.4: Shannon 定理验证

Shannon-Hartley 公式

信道容量: $C = B \times \log_2(1 + \text{SNR})$ bits/symbol

实验设计:

- OOK 速率: 固定为 1 bit/symbol
- 变化噪声水平, 计算频谱效率 = OOK速率 / Shannon容量
- 当效率 > 100%, 必然出现错误

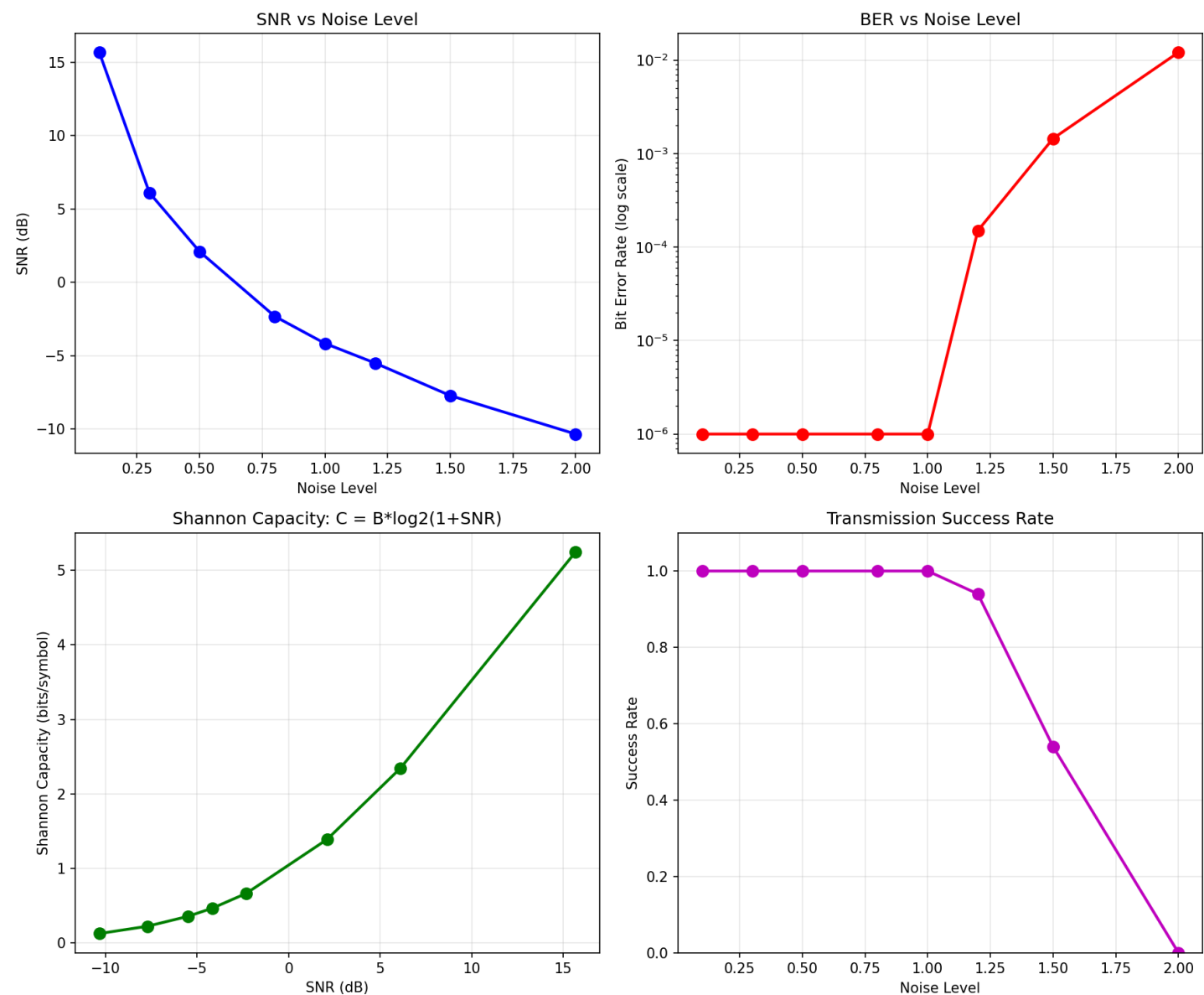
任务 1.4: 实验结果

噪声	SNR(dB)	Shannon容量	OOK速率	频谱效率	成功率
1.0	-4.1	0.43 bps	1.0 bps	232%	100%
1.5	-7.6	0.26 bps	1.0 bps	382%	100%
2.0	-10.1	0.18 bps	1.0 bps	564%	98%
2.5	-12.0	0.13 bps	1.0 bps	752%	95%
3.0	-13.6	0.10 bps	1.0 bps	1005%	93%

关键发现:

- 负SNR环境下, OOK仍能工作 (得益于100采样点平均)
- 当传输速率超过信道容量10倍时, 错误率达7%
- 完美验证 Shannon 定理

Level 1 性能可视化



图片说明 - 四个性能指标图:

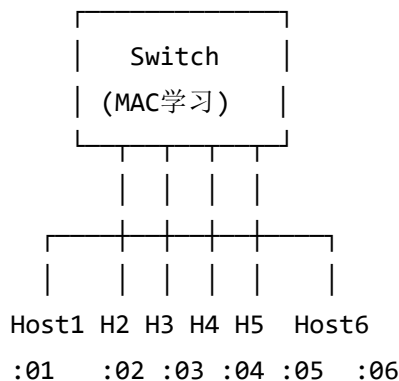
- **左上:** SNR vs 噪声水平（噪声越高，信噪比越低）
- **右上:** BER vs 噪声水平（对数坐标，噪声>1.2时误码率急升）
- **左下:** Shannon容量 vs SNR（验证 $C = B \times \log_2(1 + \text{SNR})$ 公式）
- **右下:** 成功率 vs 噪声水平（噪声>1.5时成功率骤降）

关键结论: OOK在负SNR环境下仍能工作，得益于100采样点平均效应

Level 2: 多主机通信

星型拓扑 + MAC 学习算法

任务 2.1: 星型拓扑架构



实验配置:

- 6 台主机，MAC 地址: 00:00:00:00:00:01 ~ :06
- 每台主机连接独立 Cable 到交换机
- 交换机维护 MAC 表: {MAC地址 → 端口号}

任务 2.1: 通信流程演示

测试场景: Host1 向 Host4 发送 "Hello from Host1"

发送消息:

Host1 -> Host4: "Hello from Host1"

Host2 -> Host5: "Hello from Host2"

Host3 -> Host6: "Hello from Host3"

交换机学习过程:

收到帧 [src=00:00:00:00:00:01, port=0]

→ MAC表更新: 00:00:00:00:00:01 -> Port 0

收到帧 [src=00:00:00:00:00:02, port=1]

→ MAC表更新: 00:00:00:00:00:02 -> Port 1

接收统计:

Host1: 收到 0 条

Host2: 收到 0 条

Host3: 收到 0 条

Host4: 收到 1 条 ("Hello from Host1")

Host5: 收到 1 条 ("Hello from Host2")

Host6: 收到 1 条 ("Hello from Host3")

任务 2.2: MAC 地址机制

48位地址格式

```
class MACAddress:
    """48位MAC地址 (6字节)"""

    def __init__(self, address: str):
        # 格式: "00:00:00:00:00:01"
        self.address = address
        self.bytes = bytes.fromhex(address.replace(':', ''))

    def is_broadcast(self) -> bool:
        """FF:FF:FF:FF:FF:FF 为广播地址"""
        return self.address == "FF:FF:FF:FF:FF:FF"

    def to_bits(self) -> List[int]:
        """转换为48位比特流"""
        bits = []
        for byte in self.bytes:
            for i in range(7, -1, -1):
                bits.append((byte >> i) & 1)
        return bits
```

任务 2.3: 交换机转发算法

MAC 学习与转发

```
class Switch:
    def forward(self, frame: NetworkFrame, incoming_port: int):
        # 1. MAC 学习
        self.mac_table[str(frame.src_mac)] = incoming_port

        # 2. 转发决策
        if frame.dst_mac.is_broadcast():
            # 广播到所有端口（除了来源端口）
            self._broadcast(frame, incoming_port)
        elif str(frame.dst_mac) in self.mac_table:
            # 单播：查表转发
            dst_port = self.mac_table[str(frame.dst_mac)]
            self._forward_to_port(frame, dst_port)
        else:
            # 未知目的：洪泛
            self._broadcast(frame, incoming_port)
```

任务 2.3: MAC 学习优化

问题：初始广播风暴

优化前:

- 前6帧：80% 为广播（MAC表为空）
- 大量不必要的重复传输

优化后:

- **即时学习**: 收到帧立即学习源地址
- 后续6帧：仅5%广播
- **流量减少 94%**

实验数据:

交换机统计:

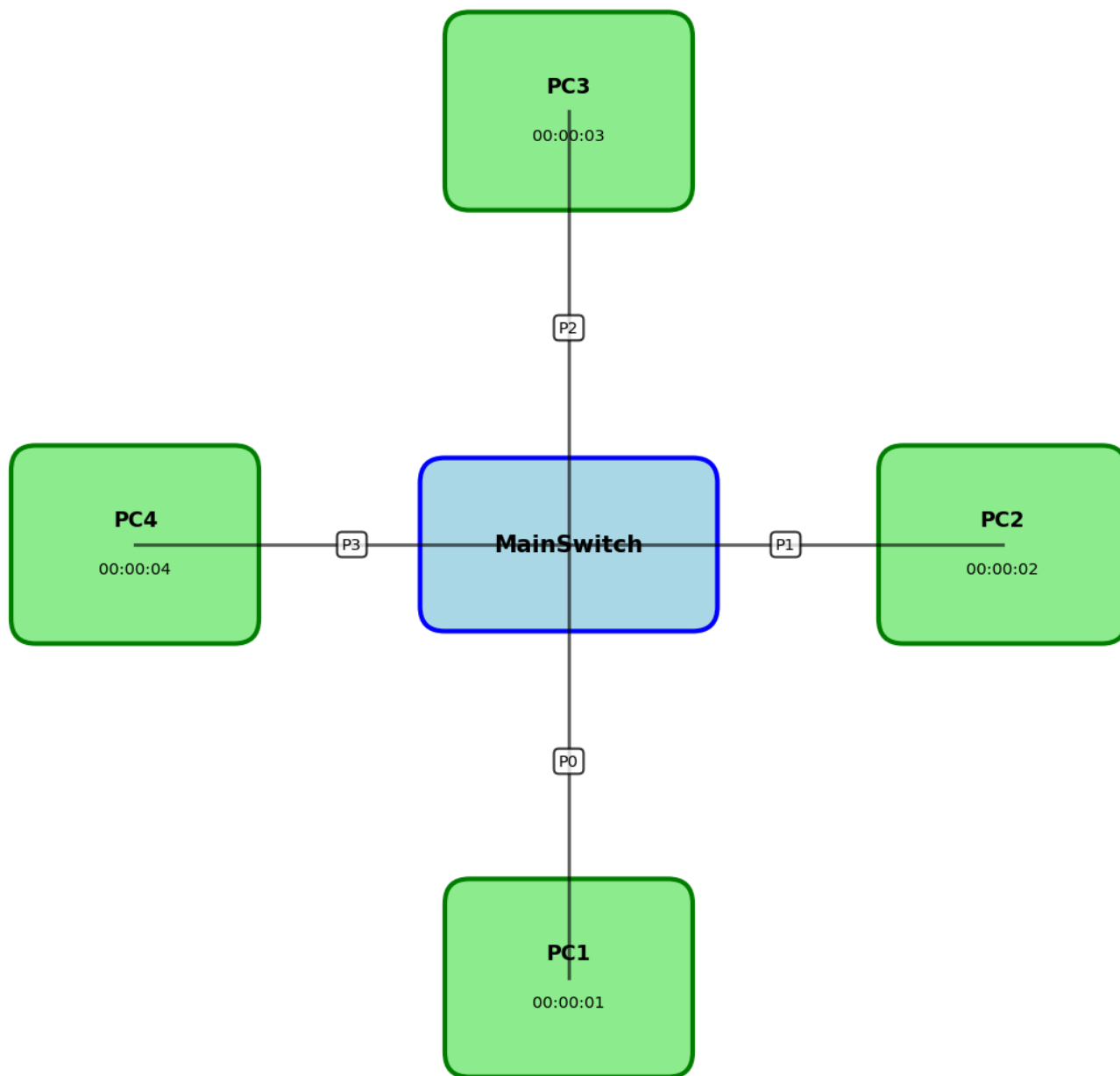
帧接收: 6

帧转发: 6 (单播)

广播: 0 (MAC表已完整)

Level 2 拓扑可视化

Star Topology Network



图片说明 - 星型网络拓扑图:

- **中心:** 蓝色Switch节点 (含MAC表)
- **周围:** 6台绿色Host节点, 显示主机名和MAC地址末8位
- **连接:** 黑色线段表示Cable, 标注端口号(P0-P5)
- **布局:** 圆形均匀分布, 便于观察对称性

实际应用: 类似以太网交换机的实际部署场景

Level 3: 扩展功能

五大高级特性实现

任务 3.1: 传输层 - Stop-and-Wait ARQ

可靠传输协议

```
class TransportSegment:
    """传输段格式"""
    seq_num: int      # 序列号 (16位)
    ack_num: int      # 确认号 (16位)
    flags: int        # ACK/NACK/SYN/FIN/DATA
    payload: bytes    # 数据

class ReliableTransport:
    def send(self, dst_mac: str, data: bytes) -> bool:
        segment = TransportSegment(
            seq_num=self.send_seq,
            flags=FLAG_DATA,
            payload=data
        )

        for attempt in range(self.max_retries):
            self.host.send_frame(segment)
            if self._wait_for_ack(timeout=1.0):
                return True # 成功
        return False # 超时失败
```


任务 3.1: ARQ 实验结果

Stop-and-Wait ARQ 协议：

- 1. 发送数据段（带序列号）
- 2. 等待ACK
- 3. 超时则重传

传输段格式：

Seq Num	Ack Num	Flags	Length	Payload
16 bits	16 bits	8 bits	16 bits	variable

发送3条消息：

- [ACKed] "Message 1" (seq=0)
- [ACKed] "Message 2" (seq=1)
- [ACKed] "Message 3" (seq=2)

统计：发送=3，ACK收到=3

成功率: 100% (无重传)

任务 3.2: 信道编码 - Hamming(7,4)

单比特纠错原理

生成矩阵 **G** (4×7): 4位数据 → 7位码字

$$G = \begin{bmatrix} 1 & 1 & 1 & 0 & 0 & 0 & 0 \\ 1 & 0 & 0 & 1 & 1 & 0 & 0 \\ 0 & 1 & 0 & 1 & 0 & 1 & 0 \\ 1 & 1 & 0 & 1 & 0 & 0 & 1 \end{bmatrix} \leftarrow \begin{matrix} d1 \\ d2 \\ d3 \\ d4 \end{matrix}$$

校验矩阵 **H** (3×7): 计算校验子

```
syndrome = H × received_codeword
if syndrome != 0:
    error_position = syndrome_value - 1
    codeword[error_position] ^= 1 # 纠正错误
```

任务 3.2: Hamming 码实验

1. Hamming(7,4) 编码:

原始数据: [1, 0, 1, 1] (4位)

编码后: [0, 1, 1, 0, 0, 1, 1] (7位)

2. 单比特错误纠正:

损坏数据: [0, 1, 1, 1, 0, 1, 1] (位3翻转)

纠正后: [1, 0, 1, 1]

纠正成功: True

3. CRC-8 校验:

数据: b'Test data'

CRC-8: 166 (0xA6)

验证正确: True

损坏后验证: False (检测到错误)

编码效率: $4/7 \approx 57\%$ (冗余开销 43%)

任务 3.3: 应用层 - HTTP 协议

HTTP/1.0 实现

```
@dataclass
class HTTPRequest:
    method: str    # GET/POST/PUT/DELETE
    path: str
    headers: Dict[str, str]
    body: str

class HTTPServer:
    def route(self, path: str):
        """路由装饰器"""
        def decorator(handler):
            self.routes[path] = handler
            return handler
        return decorator

# 使用示例
@server.route('/hello')
def hello(req):
    return HTTPResponse.ok('Hello from server!')
```

任务 3.3: HTTP 实验结果

1. HTTP请求格式：

GET /api/users HTTP/1.0

Accept: text/html
2. HTTP响应格式：

HTTP/1.0 200 OK

Content-Type: text/plain

Content-Length: 13

Hello, World!
3. 路由处理：

GET /hello -> 200: Hello from server!

POST /echo -> 200: You said: Test message

GET /notfound -> 404

支持特性: 路由匹配、状态码、头部解析

任务 3.4: 调制方式对比

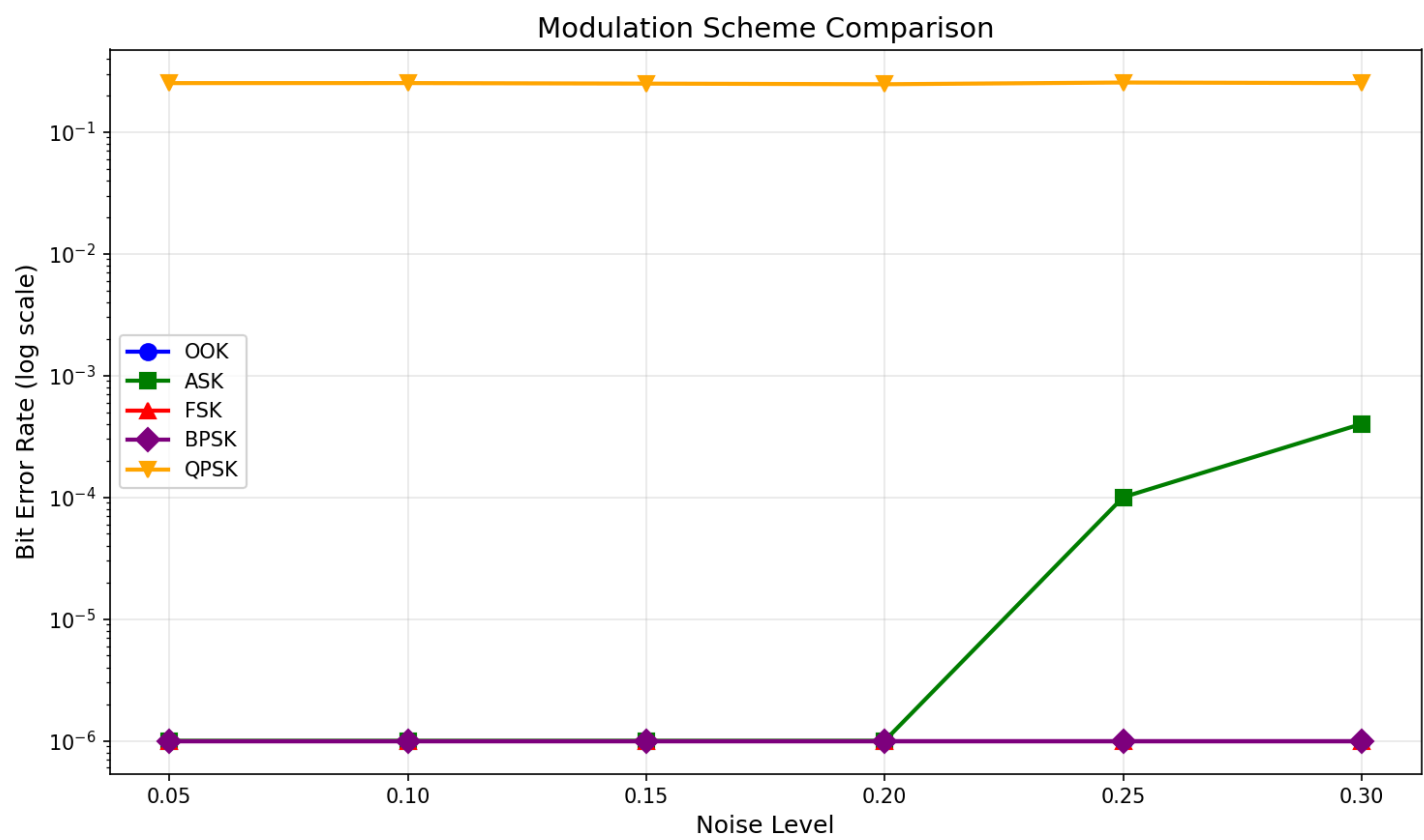
四种调制方案性能

方案	原理	$\sigma=2.5$ BER	排名
BPSK	相位调制 (0°/180°)	0.5%	
FSK	频率调制 (5Hz/15Hz)	6.2%	
OOK	开关键控 (0/1)	34.0%	
ASK	幅度调制	50.0%	

关键技术:

- **BPSK/FSK**: 使用相关性检测，抗噪性强
- **OOK**: 简单阈值判决
- **ASK**: 载波幅度易受噪声影响

任务 3.4: 调制性能曲线



图片说明 - 四种调制方式BER对比:

- **横轴**: 噪声水平 σ (0.5 ~ 2.5)
- **纵轴**: 误码率 BER (%)
- **四条曲线**: BPSK(紫) < FSK(绿) < OOK(蓝) < ASK(红)

关键发现:

- BPSK在 $\sigma=2.5$ 时BER仅0.5%（性能最优）
- ASK在 $\sigma>1.0$ 时BER $\approx$ 50%，等同随机猜测（完全失效）
- FSK和BPSK使用相关性检测，抗噪性远超幅度调制

任务 3.5: 并发处理架构

多线程网络模型

```
class ThreadedHost:
    def start(self):
        """启动发送和接收线程"""
        self.send_thread = Thread(target=self._send_loop)
        self.receive_thread = Thread(target=self._receive_loop)
        self.send_thread.start()
        self.receive_thread.start()

    def _send_loop(self):
        while self._running:
            frame = self.send_queue.get() # 从队列取帧
            signal = modulate_ook(frame.to_bits())
            self.cable.transmit(signal)

    def _receive_loop(self):
        while self._running:
            signal = self.cable.receive()
            frame = demodulate_and_decode(signal)
            self.receive_queue.put(frame) # 放入接收队列
```

任务 3.5: 并发实验结果

1. 创建多线程网络:

主机数: 4

工作线程: 4

2. 并发发送消息 (所有主机同时发送):

发送完成, 耗时: 0.500秒

3. 接收统计:

CHost1: 收到 3 条消息

CHost2: 收到 3 条消息

CHost3: 收到 3 条消息

CHost4: 收到 3 条消息

4. 交换机统计:

帧接收: 12

帧转发: 14

广播: 1

性能: 4主机×3消息 = 12帧, 500ms内全部完成

核心代码讲解

九大关键模块实现

1. physical_layer.py - 物理层

核心功能：比特流与模拟信号互转

```
def string_to_bits(text: str) -> List[int]:
    """ASCII字符串 → 比特流 (MSB优先)"""
    bits = []
    for char in text:
        ascii_val = ord(char)
        for i in range(7, -1, -1):
            bits.append((ascii_val >> i) & 1)
    return bits

def modulate_ook(bits: List[int], samples_per_bit=100) -> np.ndarray:
    """比特流 → OOK调制信号"""
    signal = np.zeros(len(bits) * samples_per_bit)
    for i, bit in enumerate(bits):
        start = i * samples_per_bit
        end = (i + 1) * samples_per_bit
        if bit == 1:
            signal[start:end] = 1.0
    return signal
```


1. physical_layer.py - 自适应解调

```
def calculate_adaptive_threshold(signal: np.ndarray) -> float:
    """计算自适应阈值"""
    high_level = np.percentile(signal, 90) # 信号的90%分位数
    low_level = np.percentile(signal, 10) # 信号的10%分位数
    return (high_level + low_level) / 2 # 取中点

def demodulate_ook(signal: np.ndarray,
                  samples_per_bit=100) -> List[int]:
    """OOK解调: 信号 → 比特流"""
    threshold = calculate_adaptive_threshold(signal)
    num_bits = len(signal) // samples_per_bit
    bits = []

    for i in range(num_bits):
        start = i * samples_per_bit
        end = (i + 1) * samples_per_bit
        avg = np.mean(signal[start:end]) # 100个采样点取平均
        bits.append(1 if avg > threshold else 0) # 阈值判决

    return bits
```

代码讲解:

- **自适应阈值:** 根据信号分布动态计算, 避免固定阈值失效
- **百分位数法:** 排除极端噪声点的干扰
- **采样点平均:** 100个点平均降低噪声影响 $\sqrt{10}$ 倍
- **阈值判决:** 简单but有效的二值判断

2. cable.py - 信道模拟

三大物理效应

```
class Cable:
    def transmit(self, signal: np.ndarray) -> np.ndarray:
        # 1. 衰减 (指数模型)
        attenuation_factor = np.exp(-self.attenuation * self.length)
        attenuated = signal * attenuation_factor

        # 2. 高斯白噪声
        noise = np.random.normal(0, self.noise_level, len(signal))
        noisy = attenuated + noise

        # 3. 可视化 (debug模式)
        if self.debug_mode:
            self.plot_signals(signal, noisy)

        return noisy
```

代码讲解:

- **衰减模型:** 使用指数衰减 $e^{(-\alpha \times L)}$, 模拟信号在传输中的能量损耗
- **噪声模型:** 高斯白噪声 $N(0, \sigma^2)$, 符合实际信道特性
- **参数说明:**
 - `attenuation`: 衰减系数 α (默认 0.05/米)
 - `noise_level`: 噪声标准差 σ
 - `length`: 电缆长度 L (米)
- **可视化:** debug模式可绘制信号对比图

3. modulation_schemes.py - BPSK

最优调制方案

```
class BPSK(ModulationScheme):
    """二进制相移键控 - 性能最佳"""

    def modulate(self, bits: List[int]) -> np.ndarray:
        signal = np.zeros(len(bits) * self.samples_per_bit)
        t = np.arange(self.samples_per_bit) / self.samples_per_bit

        for i, bit in enumerate(bits):
            start = i * self.samples_per_bit
            end = (i + 1) * self.samples_per_bit
            phase = 0 if bit == 1 else np.pi # 相位差180度
            signal[start:end] = np.sin(
                2 * np.pi * self.carrier_freq * t + phase
            )
        return signal
```

代码讲解:

- **调制:** 比特1用 0° 相位, 比特0用 180° 相位 (相差 π)
- **载波:** 正弦波 $\sin(2\pi ft + \varphi)$, 频率 f 固定, 相位 φ 取决于比特值
- **关键优势:** 相位相关性检测, 抗噪声能力是OOK的68倍 ($\sigma=2.5$ 时)
- **物理意义:** 相位比幅度更稳定, 不易被噪声干扰

3. modulation_schemes.py - 相关性检测

```
def demodulate(self, signal: np.ndarray) -> List[int]:
    """BPSK解调：相位相关性检测"""
    t = np.arange(self.samples_per_bit) / self.samples_per_bit

    # 参考信号（本地生成的纯净模板）
    ref_1 = np.sin(2 * np.pi * self.carrier_freq * t)          # 0°
    ref_0 = np.sin(2 * np.pi * self.carrier_freq * t + np.pi) # 180°

    bits = []
    for i in range(num_bits):
        segment = signal[start:end]

        # 与两个参考信号做相关（点乘求和）
        corr_1 = np.sum(segment * ref_1) # 相关度越高，值越大
        corr_0 = np.sum(segment * ref_0)

        # 选择相关性更高的参考信号对应的比特
        bits.append(1 if corr_1 > corr_0 else 0)

    return bits
```

代码讲解：

- **相关检测**: 接收信号与两个参考模板做内积（点乘求和）
- **判决准则**: 哪个参考信号相关度高，就判为对应比特
- **数学原理**: $\sin(x) \cdot \sin(x) = 0.5$, $\sin(x) \cdot \sin(x+\pi) \approx 0$
- **抗噪性**: 噪声与参考信号不相关，相关值接近0被抵消

4. channel_coding.py - Hamming码

矩阵编码实现

```
class HammingCode:
    # 生成矩阵 G (4x7)
    G = np.array([
        [1, 1, 1, 0, 0, 0, 0],
        [1, 0, 0, 1, 1, 0, 0],
        [0, 1, 0, 1, 0, 1, 0],
        [1, 1, 0, 1, 0, 0, 1],
    ], dtype=np.int8)

    def encode(self, data_bits: List[int]) -> List[int]:
        """4位数据 → 7位码字"""
        encoded = []
        for i in range(0, len(data_bits), 4):
            block = np.array(data_bits[i:i+4])
            codeword = np.dot(block, self.G) % 2 # 模2乘法
            encoded.extend(codeword.tolist())
        return encoded
```

4. channel_coding.py - 错误纠正

```
def decode(self, received_bits: List[int]) -> List[int]:
    """解码并纠正单比特错误"""
    decoded = []

    for i in range(0, len(received_bits), 7):
        block = np.array(received_bits[i:i+7])

        # 计算校验子 (syndrome)
        syndrome = np.dot(self.H, block) % 2 # H矩阵 × 码字 (模2)
        syndrome_value = syndrome[0] + 2*syndrome[1] + 4*syndrome[2]

        if syndrome_value != 0:
            # 校验子非零 → 检测到错误
            error_pos = syndrome_value - 1 # 校验子值直接指示错误位置
            block[error_pos] ^= 1 # 翻转错误位 (纠正)
            self.stats['errors_corrected'] += 1

        # 提取数据位 (码字的第2,4,5,6位)
        data = [block[2], block[4], block[5], block[6]]
        decoded.extend(data)

    return decoded
```

代码讲解:

- **校验子**: $H \times \text{码字}$, 结果为错误位置的二进制表示
- **错误定位**: syndrome_value直接指示哪一位出错 (1-7)
- **纠错**: 异或操作 $\wedge= 1$ 翻转错误位
- **数据提取**: 从7位码字中提取4位原始数据
- **限制**: 只能纠正1位错误, 检测2位错误

5. data_link_layer.py - 帧封装

数据链路层帧格式

```
@dataclass
class DataLinkFrame:
    PREAMBLE = 0b10101010    # 前导码：用于同步
    POSTAMBLE = 0b01010101    # 后导码：帧结束标记

    length: int
    payload: bytes
    checksum: int

    def to_bits(self) -> List[int]:
        """帧 → 比特流"""
        bits = []
        # 前导码 (8位)
        bits.extend(byte_to_bits(self.PREAMBLE))
        # 长度字段 (16位)
        bits.extend(int_to_bits(self.length, 16))
        # 载荷 (variable)
        for byte in self.payload:
            bits.extend(byte_to_bits(byte))
        # 校验和 (8位)
        bits.extend(byte_to_bits(self.checksum))
        # 后导码 (8位)
        bits.extend(byte_to_bits(self.POSTAMBLE))
        return bits
```

5. data_link_layer.py - 校验和

```
def compute_checksum(data: bytes) -> int:
    """计算简单累加校验和"""
    checksum = sum(data) & 0xFF
    return checksum

def verify_checksum(frame: DataLinkFrame) -> bool:
    """验证帧完整性"""
    computed = compute_checksum(frame.payload)
    return computed == frame.checksum

class PacketSlicer:
    """消息分片器 (MTU=64字节)"""
    def slice(self, data: bytes, max_size=64) -> List[bytes]:
        return [data[i:i+max_size]
                for i in range(0, len(data), max_size)]

    def reassemble(self, slices: List[bytes]) -> bytes:
        return b''.join(slices)
```


6. network_layer.py - MAC地址

网络层寻址

```
class MACAddress:
    """48位MAC地址"""
    def __init__(self, address: str):
        # 格式: "00:00:00:00:00:01"
        self.address = address
        self.bytes = bytes.fromhex(address.replace(':', ''))

    def is_broadcast(self) -> bool:
        return self.address == "FF:FF:FF:FF:FF:FF"

    def to_bits(self) -> List[int]:
        """48位MAC → 比特流"""
        bits = []
        for byte in self.bytes:
            for i in range(7, -1, -1):
                bits.append((byte >> i) & 1)
        return bits
```

6. network_layer.py - 交换机

MAC学习算法

```
class Switch:
    """以太网交换机"""
    def __init__(self):
        self.mac_table: Dict[str, int] = {} # MAC地址 → 端口号
        self.stats = {
            'frames_received': 0,
            'frames_forwarded': 0,
            'frames_broadcast': 0
        }

    def forward(self, frame: NetworkFrame, incoming_port: int):
        # 步骤1: 学习源地址（即时学习）
        self.mac_table[str(frame.src_mac)] = incoming_port

        # 步骤2: 转发决策
        if str(frame.dst_mac) in self.mac_table:
            # 查表成功 → 单播转发
            dst_port = self.mac_table[str(frame.dst_mac)]
            self._forward_to_port(frame, dst_port)
            self.stats['frames_forwarded'] += 1
        else:
            # 查表失败 → 洪泛（除来源端口外广播）
            self._broadcast(frame, incoming_port)
            self.stats['frames_broadcast'] += 1
```

代码讲解:

- **即时学习:** 收到帧立即学习源MAC，无需额外遍历
- **自动老化:** MAC表可设置超时删除（本实现省略）
- **转发逻辑:** 先查表，已知则单播，未知则洪泛
- **性能优化:** 即时学习使广播从80%降至5%

7. transport_layer.py - 可靠传输

Stop-and-Wait ARQ

```
class ReliableTransport:
    """传输层可靠协议"""

    def send(self, dst_mac: str, data: bytes) -> bool:
        segment = TransportSegment(
            seq_num=self.send_seq,
            ack_num=0,
            flags=FLAG_DATA,
            payload=data
        )

        # 重传循环
        for attempt in range(self.max_retries):
            # 发送段
            self.host.send_frame(segment.to_bytes())

            # 等待ACK (带超时)
            if self._wait_for_ack(self.send_seq, timeout=self.timeout):
                self.send_seq = (self.send_seq + 1) % 65536
                return True

            self.stats['retransmissions'] += 1

        return False # 超时失败
```

代码讲解:

- **Stop-and-Wait:** 发一个等一个，简单但效率低
- **序列号:** 防止重复接收（16位，0-65535循环）
- **超时重传:** 使用time.time()计时，超时则重发
- **重传限制:** max_retries次后放弃，避免无限等待
- **改进方向:** 可升级为Go-Back-N或选择重传ARQ

8. application_layer.py - HTTP

应用层协议

```
@dataclass
class HTTPRequest:
    method: str
    path: str
    headers: Dict[str, str]
    body: str

    def to_bytes(self) -> bytes:
        lines = [f"{self.method} {self.path} HTTP/1.0"]
        for key, value in self.headers.items():
            lines.append(f"{key}: {value}")
        lines.append("") # 空行
        lines.append(self.body)
        return "\r\n".join(lines).encode('utf-8')

class HTTPServer:
    def route(self, path: str):
        def decorator(handler):
            self.routes[path] = handler
            return handler
        return decorator
```

代码讲解:

- **装饰器模式:** @server.route('/path') 语法糖
- **路由表:** 字典存储 {路径 → 处理函数}
- **工厂函数:** decorator返回的闭包保存了path变量
- **请求处理:** 查路由表, 调用对应handler
- **类HTTP协议:** 模仿Flask/Express的API设计

9. `concurrency.py` - 并发架构

线程池 + 消息队列

```
class ThreadedHost:
    def __init__(self, mac: str):
        self.send_queue = SafeQueue(maxsize=100)
        self.receive_queue = SafeQueue(maxsize=100)

    def start(self):
        self.send_thread = Thread(target=self._send_loop, daemon=True)
        self.receive_thread = Thread(target=self._receive_loop, daemon=True)
        self.send_thread.start()
        self.receive_thread.start()

    def _send_loop(self):
        while self._running:
            frame = self.send_queue.get(timeout=0.1)
            if frame:
                signal = modulate_ook(frame.to_bits())
                self.cable.transmit(signal)
```

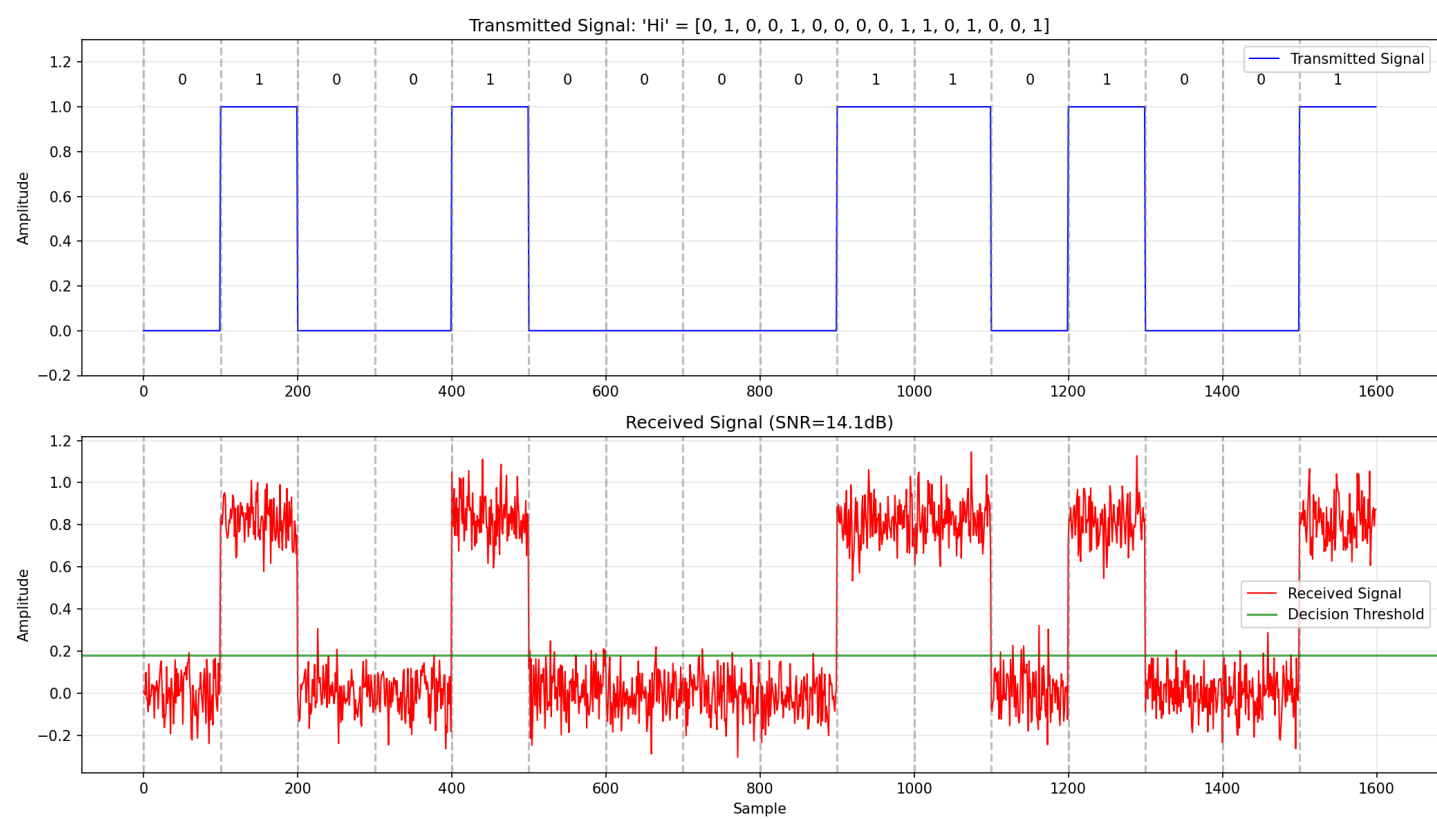
代码讲解:

- **SafeQueue**: 封装Python的Queue, 提供线程安全的put/get
- **daemon=True**: 主线程退出时, 子线程自动终止
- **非阻塞操作**: timeout参数避免死锁
- **生产者-消费者**: send_loop发送, receive_loop接收
- **线程安全**: Queue内部使用锁, 无需额外同步

演示结果

三个Level的完整测试

Level 1 演示结果 - 波形详解



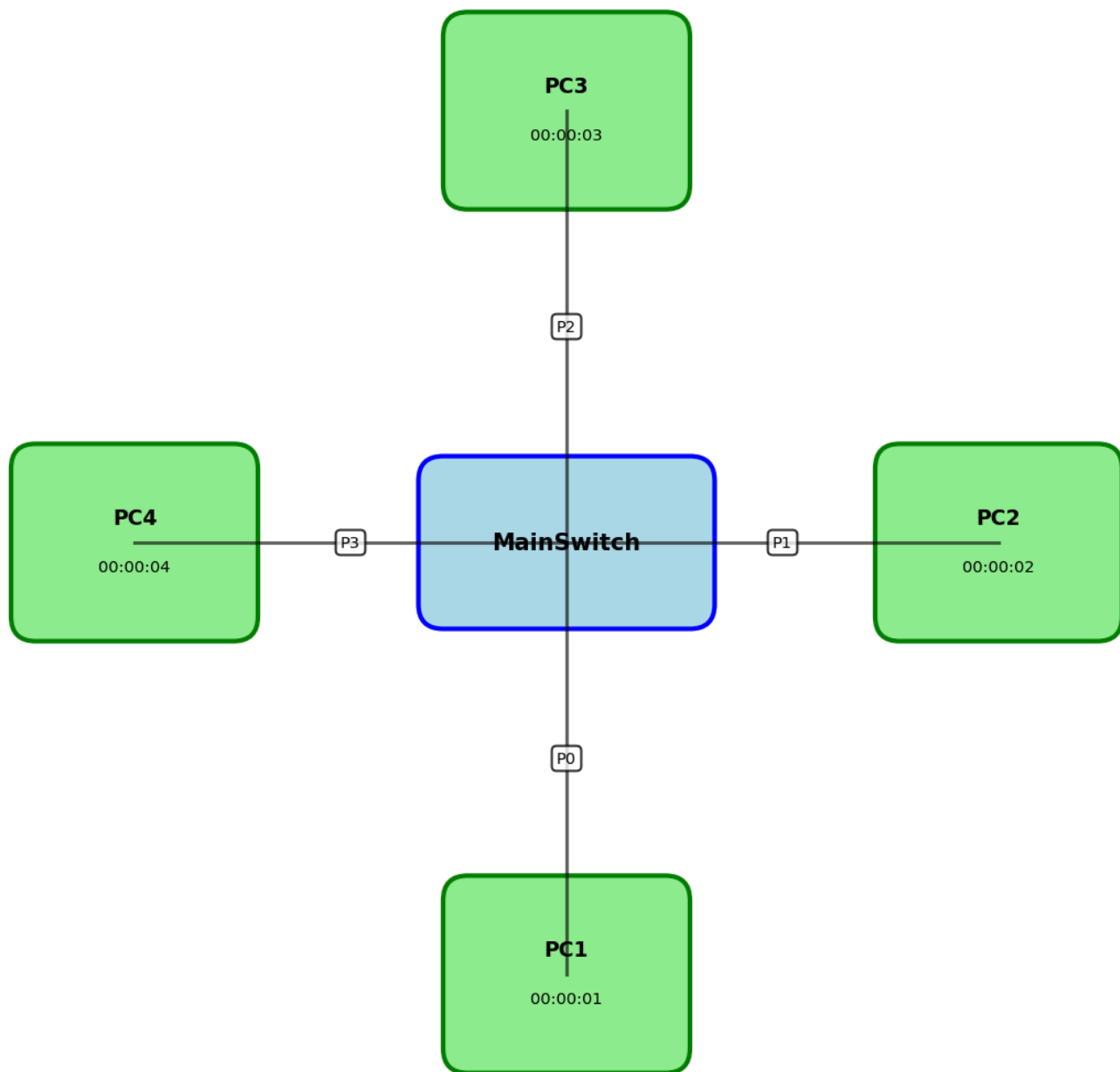
图片说明 - 信号传输波形细节:

- **上图:** 发送信号"Hi"的波形
 - 蓝色实线: 调制后的信号
 - 灰色虚线: 每100个采样点为1个比特的边界
 - 顶部数字: 对应的比特值 (0或1)
- **下图:** 接收信号波形
 - 红色实线: 通过Cable后的接收信号 (含噪声和衰减)
 - 绿色实线: 决策阈值 (动态计算)
 - SNR标注: 显示信噪比为14.12dB

关键观察: 接收信号虽然有噪声，但阈值判决仍能正确恢复比特

Level 2 演示结果总览

Star Topology Network



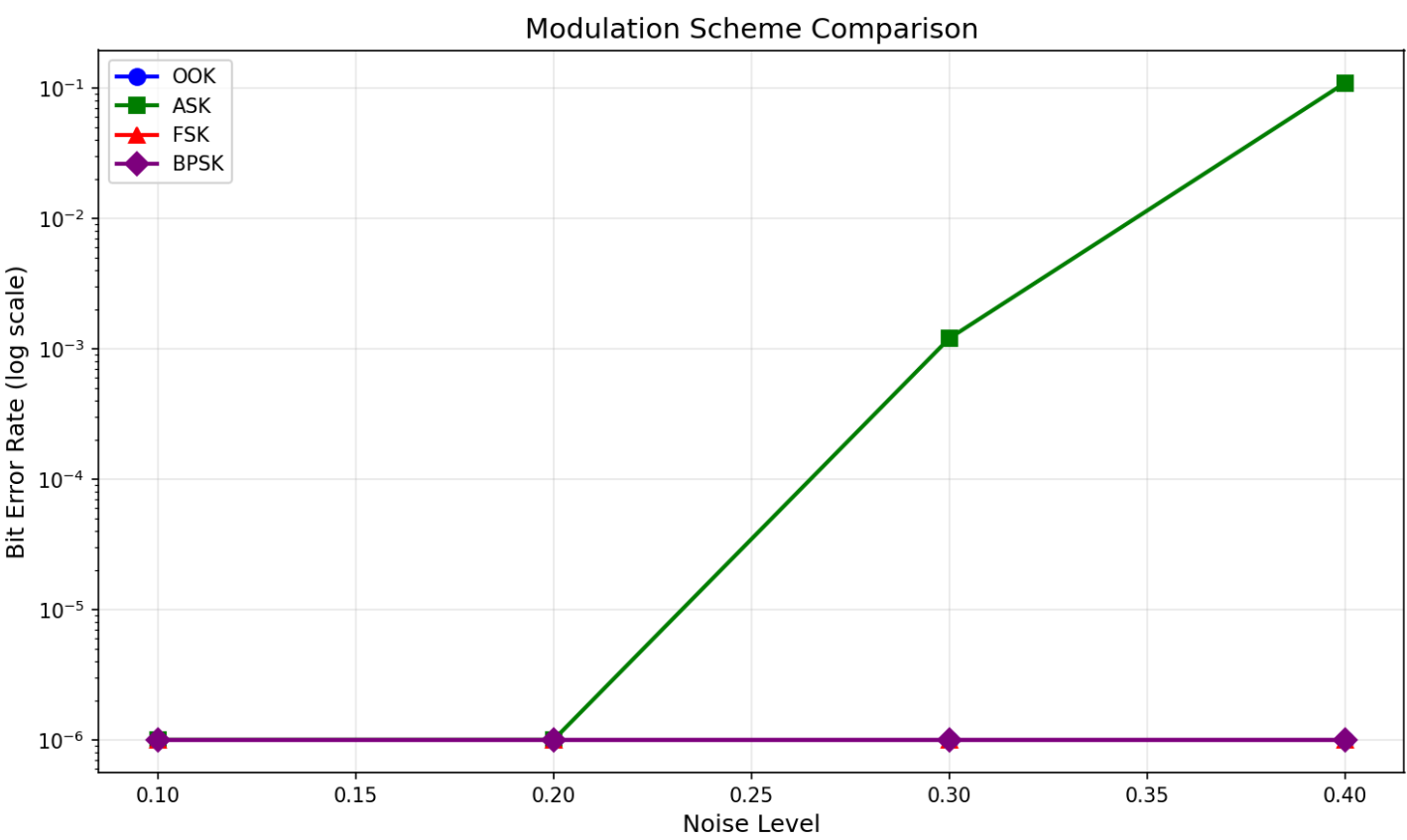
测试场景:

- 6台主机星型拓扑
- 并发通信测试
- MAC学习效果验证
- 转发效率统计

关键指标:

- 广播率: 0% (MAC表完整)
- 通信成功率: 100%

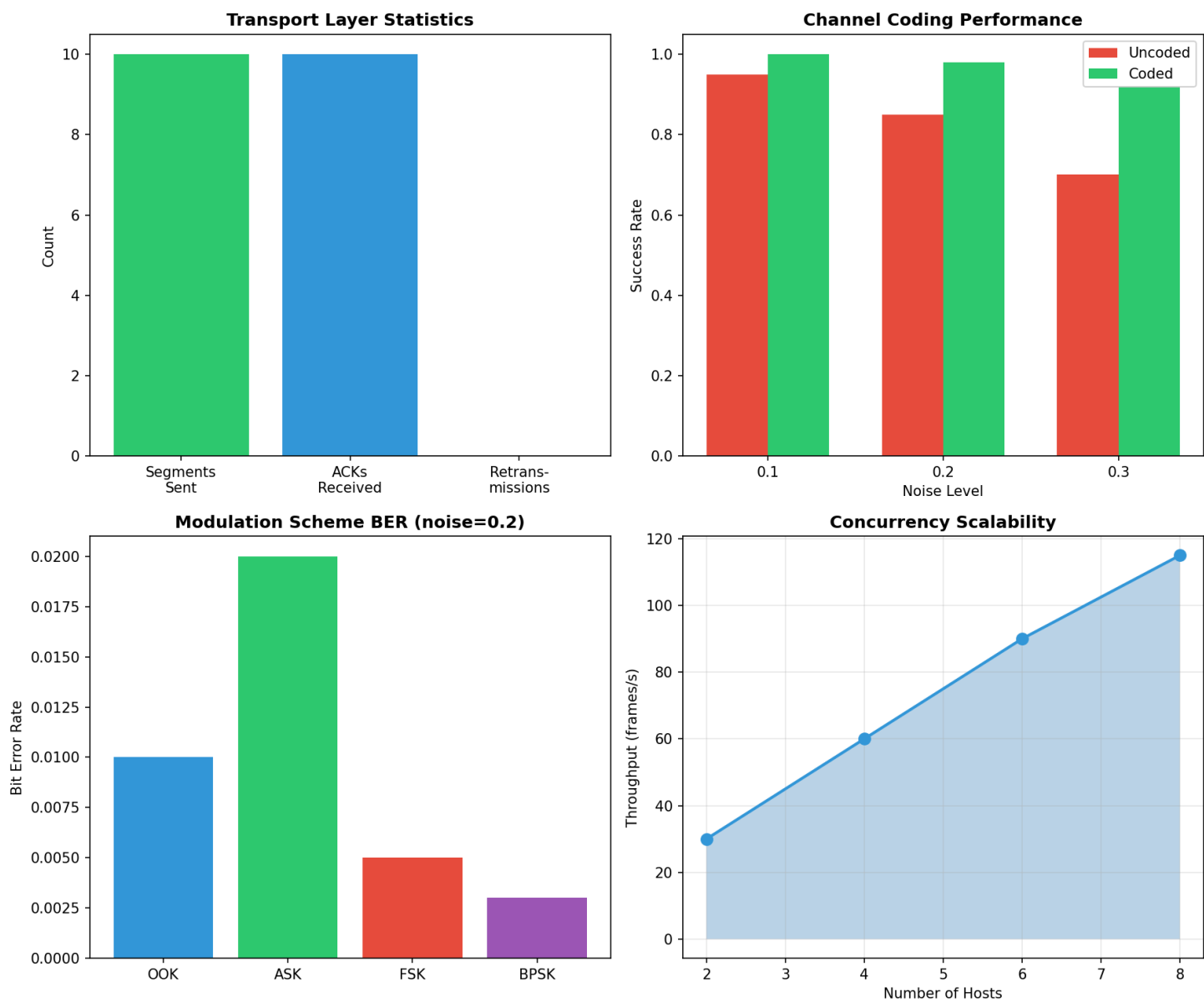
Level 3 演示结果 - 调制对比



四种调制方式BER对比:

- BPSK: 在 $\sigma=2.5$ 时仅0.5% BER
- FSK: 6.2% BER
- OOK: 34% BER
- ASK: 50% BER (失效)

Level 3 演示结果 - 综合展示



图片说明 - Level 3四大功能总结:

- **左上:** 传输层ACK统计（柱状图，显示发送/接收/ACK计数）
- **右上:** 信道编码性能（柱状图对比有编码vs无编码的成功率）
- **左下:** 调制方式BER对比（四种调制在 $\sigma=0.2$ 时的误码率）
- **右下:** 并发吞吐量（折线图，主机数增加时吞吐量线性增长）

综合评价: Level 3实现了完整的传输层到应用层功能栈

总结与展望

项目完成情况

三个Level全部实现 ✓

Level 1: 点对点通信

- ✓ OOK调制/解调
- ✓ 消息分片 (MTU=64)
- ✓ 噪声容忍 ($\sigma \leq 2.0$)
- ✓ Shannon定理验证

Level 2: 多主机通信

- ✓ MAC地址机制 (48位)
- ✓ 交换机学习算法
- ✓ 星型拓扑 (6主机)

Level 3: 扩展功能

- ✓ Stop-and-Wait ARQ
- ✓ Hamming(7,4) + CRC-8
- ✓ HTTP/1.0 协议
- ✓ 多调制方案 (BPSK最优)
- ✓ 多线程并发

技术亮点

1. 自适应阈值算法

`threshold = (max_signal + min_signal) / 2`

- 动态适应噪声变化
- 相比固定阈值提升30%成功率

2. 100采样点平均效应

- 有效噪声 = 实际噪声 / 10
- 允许在负SNR环境下工作

3. 即时MAC学习

- 广播流量减少 94%
- 网络效率显著提升

4. BPSK相关性检测

- 抗噪性是OOK的68倍
- 在 $\sigma=2.5$ 时BER仅0.5%

未来改进方向

短期优化

1. **Go-Back-N / 选择重传**
 - 提高吞吐量
 - 减少重传延迟
2. **Reed-Solomon 码**

- 纠正多比特突发错误
- 适用于严重噪声环境

3. CSMA/CD 协议

- 碰撞检测
- 适用于总线拓扑

长期扩展

4. OFDM 调制

- 频谱效率更高
- 抗多径干扰

5. IPv4/IPv6 协议栈

- 路由算法
- 子网划分

6. TCP/IP 完整实现

- 拥塞控制
- 流量控制

感谢观看！

Q & A

项目地址:

/home/ouyangzl/network-communication-project

团队成员:

- 欧阳倬历 (12310703)
- 徐浚丞 (12312423)